# Contents

**Infrastructure as Code — Terraform, Ansible & Beyond** **1**
1. Terraform Deep Dive . . . 1
Core Concepts . . . 1
HCL Syntax Essentials . . . 1
Terraform State Management . . . 3
Workspace Pattern . . . 3
Lifecycle Rules . . . 3
2. Terraform Modules . . . 3
Module Structure . . . 3
Module Outputs . . . 4
3. Terraform Testing & Validation . . . 4
4. Ansible Deep Dive . . . 5
Inventory . . . 5
Playbook . . . 5
Role Structure . . . 6
Ansible Vault . . . 6
Advanced Features . . . 7
Molecule — Testing Roles . . . 7
5. Terraform vs Ansible vs Pulumi . . . 8
6. Terragrunt (Terraform DRY) . . . 8
 Resources . . . 8

# Infrastructure as Code — Terraform, Ansible & Beyond

## 1. Terraform Deep Dive

### Core Concepts

**Providers** — plugins that interact with APIs (AWS, GCP, Kubernetes, GitHub) **Resources** — infrastructure objects managed by Terraform **Data Sources** — read existing infrastructure (not managed by TF) **Variables** — inputs; **Outputs** — exports **State** — JSON file tracking real-world resource mappings **Modules** — reusable groups of resources

### HCL Syntax Essentials

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "instance_config" {
  type = object({
    instance_type  = string
    ami_id         = string
    min_size       = number
    max_size       = number
    desired_size   = number
  })
  default = {
    instance_type  = "t3.medium"
    ami_id         = "ami-0abcdef1234567890"
    min_size       = 2
```

```
    max_size       = 10
    desired_size   = 3
  }
}

# locals.tf — computed values
locals {
  name_prefix  = "${var.project}-${var.environment}"
  common_tags  = {
    Environment = var.environment
    Project     = var.project
    ManagedBy   = "terraform"
    Owner       = "platform-team"
  }
}

# main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-vpc" })
}

# Data source — read existing resource
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# Dynamic blocks
resource "aws_security_group" "web" {
  name   = "${local.name_prefix}-web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.allowed_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}

# count and for_each
resource "aws_subnet" "private" {
  count             = length(var.private_subnet_cidrs)
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidrs[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-private-${count.index}" })
}

# for_each — preferred over count for complex resources
```

```
resource "aws_iam_user" "team" {
  for_each = toset(var.team_members)
  name     = each.value
  tags     = local.common_tags
}
```

#### Terraform State Management

```
# Remote backend — ALWAYS use in teams
terraform {
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "production/eks/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"  # prevents concurrent runs
  }
  required_version = ">= 1.7.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"  # pessimistic constraint
    }
  }
}
```

#### Workspace Pattern

```
terraform workspace new staging
terraform workspace new production
terraform workspace select production
terraform apply -var-file="environments/production.tfvars"
```

#### Lifecycle Rules

```
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy       = true          # block terraform destroy
    ignore_changes        = [password]    # don't track password changes (set externally)
    create_before_destroy = true          # blue-green for stateful resources
  }
}
```

---

## 2. Terraform Modules

#### Module Structure

```
modules/
└── eks-cluster/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    └── versions.tf
```

```
# Usage
module "eks" {
  source  = "./modules/eks-cluster"
  # or from registry:
  # source  = "terraform-aws-modules/eks/aws"
  # version = "~> 20.0"

  cluster_name    = "${local.name_prefix}-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  tags            = local.common_tags
}
```

**Module Outputs**

```
# outputs.tf in module
output "cluster_endpoint" {
  description = "EKS cluster API endpoint"
  value       = aws_eks_cluster.this.endpoint
}

output "cluster_certificate_authority_data" {
  description = "Base64 encoded CA cert"
  value       = aws_eks_cluster.this.certificate_authority[0].data
  sensitive   = true
}

# Reference from root
resource "helm_release" "argocd" {
  set {
    name  = "server.ingress.hostname"
    value = module.eks.cluster_endpoint
  }
}
```

---

## 3. Terraform Testing & Validation

```
# Format check
terraform fmt -check -recursive

# Lint with tflint
tflint --init
tflint --recursive

# Static analysis
checkov -d . --framework terraform
tfsec .

# Plan and save
terraform plan -out=tfplan
terraform show -json tfplan | jq '.resource_changes'

# Cost estimation
infracost breakdown --path . --format table

# Full automated test with Terratest (Go)
```

```go
# test/eks_test.go
func TestEKSCluster(t *testing.T) {
    opts := &terraform.Options{
        TerraformDir: "../",
        Vars: map[string]interface{}{
            "environment": "test",
        },
    }
    defer terraform.Destroy(t, opts)
    terraform.InitAndApply(t, opts)

    endpoint := terraform.Output(t, opts, "cluster_endpoint")
    assert.NotEmpty(t, endpoint)
}
```

---

## 4. Ansible Deep Dive

### Inventory

```ini
# inventory/production.ini
[webservers]
web1 ansible_host=10.0.1.10 ansible_user=ec2-user
web2 ansible_host=10.0.1.11 ansible_user=ec2-user

[databases]
db1 ansible_host=10.0.2.10 ansible_user=ec2-user

[production:children]
webservers
databases

[production:vars]
ansible_ssh_private_key_file=~/.ssh/prod_key.pem
env=production
```

### Playbook

```yaml
---
- name: Configure web servers
  hosts: webservers
  become: true
  gather_facts: true
  vars:
    app_version: "{{ lookup('env', 'APP_VERSION') | default('latest') }}"
    nginx_worker_processes: "{{ ansible_processor_vcpus }}"

  pre_tasks:
    - name: Update package cache
      ansible.builtin.package:
        update_cache: true
      when: ansible_os_family == "Debian"

  roles:
    - common
    - nginx
    - app-deploy
```

```yaml
  tasks:
    - name: Deploy application config
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /etc/app/config.conf
        owner: app
        group: app
        mode: "0640"
      notify: Restart app

    - name: Ensure service is running
      ansible.builtin.systemd:
        name: myapp
        state: started
        enabled: true

  handlers:
    - name: Restart app
      ansible.builtin.systemd:
        name: myapp
        state: restarted
```

**Role Structure**

```
roles/nginx/
├── tasks/
│   ├── main.yml
│   ├── install.yml
│   └── configure.yml
├── handlers/
│   └── main.yml
├── templates/
│   └── nginx.conf.j2
├── files/
│   └── ssl/
├── vars/
│   └── main.yml
├── defaults/
│   └── main.yml          # lowest priority, overridable
└── meta/
    └── main.yml          # role dependencies
```

**Ansible Vault**

```yaml
# Encrypt file
ansible-vault encrypt vars/secrets.yml

# Use in playbook
vars_files:
  - vars/main.yml
  - vars/secrets.yml   # encrypted

# Run with vault password
ansible-playbook site.yml --vault-password-file ~/.vault_pass
# or: --ask-vault-pass

# Encrypt single value inline
ansible-vault encrypt_string 'supersecret' --name 'db_password'
```

### Advanced Features

```yaml
# Blocks with error handling
- block:
    - name: Risky operation
      ansible.builtin.command: /risky/command
  rescue:
    - name: Handle failure
      ansible.builtin.debug:
        msg: "Failed, running recovery"
  always:
    - name: Always cleanup
      ansible.builtin.file:
        path: /tmp/workdir
        state: absent

# Delegation — run task on different host
- name: Remove from load balancer
  ansible.builtin.uri:
    url: "http://lb.example.com/api/remove/{{ inventory_hostname }}"
    method: POST
  delegate_to: localhost

# Wait for condition
- name: Wait for app to be ready
  ansible.builtin.uri:
    url: "http://{{ inventory_hostname }}:8080/health"
    status_code: 200
  register: result
  until: result.status == 200
  retries: 30
  delay: 10
```

### Molecule — Testing Roles

```yaml
# molecule/default/molecule.yml
driver:
  name: docker
platforms:
  - name: instance
    image: ubuntu:22.04
    pre_build_image: true

# molecule/default/converge.yml
- name: Converge
  hosts: all
  roles:
    - role: my-role

# molecule/default/verify.yml
- name: Verify
  hosts: all
  tasks:
    - name: Check nginx is running
      ansible.builtin.service_facts:
    - assert:
        that: "'nginx' in services and services['nginx']['state'] == 'running'"
```

## 5. Terraform vs Ansible vs Pulumi

| Aspect | Terraform | Ansible | Pulumi |
|---|---|---|---|
| Language | HCL (declarative) | YAML (imperative/declarative) | Python/TypeScript/Go |
| State | Managed (tfstate) | Stateless | Managed (Pulumi backend) |
| Best for | Cloud infra provisioning | Config management, app deploy | Infra as real code |
| Idempotency | Yes (plan → apply) | Yes (with idempotent modules) | Yes |
| Mutable infra | Replace resources | Configures in-place | Replace resources |
| Learning curve | Moderate | Low-Medium | High (requires coding) |

**Rule of thumb:** - Terraform → provision cloud resources (VPCs, EKS, RDS, S3) - Ansible → configure OS, install software, deploy apps to VMs - Helm/Kustomize → deploy apps to Kubernetes

---

## 6. Terragrunt (Terraform DRY)

```
# terragrunt.hcl (root)
remote_state {
  backend = "s3"
  config = {
    bucket = "company-tf-state"
    key    = "${path_relative_to_include()}/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "terraform-state-lock"
  }
}

# environments/prod/vpc/terragrunt.hcl
include "root" {
  path = find_in_parent_folders()
}

terraform {
  source = "../../../modules//vpc"
}

inputs = {
  environment = "prod"
  cidr_block  = "10.0.0.0/16"
}
```

## Resources

- https://developer.hashicorp.com/terraform/docs — Official Terraform docs
- https://www.terraform-best-practices.com/ — Best practices guide
- https://docs.ansible.com/ — Ansible docs
- https://molecule.readthedocs.io/ — Molecule testing
- https://www.pulumi.com/docs/ — Pulumi docs
- https://terragrunt.gruntwork.io/ — Terragrunt
- https://www.gruntwork.io/guides — Infrastructure best practices